

Assignment 3

Please submit the code and report (in pdf format) on Blackboard system before 23:59 May 24. Report can be written either in English or Chinese. Name your report as

`studentID_Name.pdf`

Q1

Read the data from json file `/shareddata/data/iot-devices/iot_devices.json`.

Complete the following tasks **using Spark Streaming API**.

(1) [1 point] For each `device_name`, extract its type by the former names (e.g., 'meter-gauge-409Jn3BxZ2LE' -> 'meter-gauge'). Group by `device_type` and `cn`, count each group. Sort the result by `cn` in descending order. Use the event-time (`timestamp`) window of 1 second (with zero overlap of adjacent windows). Write the output to parquet file with append mode.

(2) [1 point] Group the streaming data by `cn` and aggregate each group by the number of devices which have battery level equal to or lower than 5. Use the event-time (`timestamp`) window of 1 second (with zero overlap of adjacent windows). Set the checkpoint location path. Write the output to parquet file with append mode.

Note:

- You can design a reasonable watermark time by yourself.
- You can try to split the original json data into multiple files and set `maxFilesPerTrigger` as `1` to observe the streaming processing behavior.
- You can also try to simulate the streaming data in the wild by kafka producer. You can set a reasonable sending interval according to the `timestamp` information by yourself.
- The `timestamp` column is the timestamp in milliseconds, you can use `timestamp / 1000` to get the timestamp in seconds. More introduction about unix timestamp is [here](#).

Q2

Use the data in `/shareddata/data/activity-data/` path. Complete the following tasks using **Spark Streaming API**.

(1) [2 points] Read the streaming data from json files. Set the partitions of the streaming data as 3 with Spark configuration. Count the streaming data grouped by `user` with an event-time (`Creation_Time`) window of 6 minutes. The overlap of adjacent windows is 3 minutes. Write the streaming data output into memory with update mode. Name the query as `activity_query` and configure the checkpoint location path. Display the first 3 query results of `activity_query` .

(2) [1 point] For the above task, update your code to write the query results simultaneously into memory and parquet files with append mode.

Q3

The house pricing data is attached. Process the data and split the `train.csv` to train data (80%) and validation data (20%). Tune model hyperparameters with validation data. Write the predictions of the house price in the test set to a file `predictions.csv` (with columns of `id` and `SalePrice`). You can use all or part of features in the data.

- (1) [1 points] Predict with decision tree using Pyspark MLlib package.
- (2) [2 points] Predict with [XGBoost](#) using pyspark platform.
- (3) [2 points] Predict with [LightGBM](#) using pyspark platform.

Note:

1. Assume the data is very large so that it has to be preprocessed and modeled by pyspark (Both Dataframe API and RDD are OK).
2. You can refer to the [XGBoost documentation](#) and [SynapseML lightGBM Toolkit](#).